

C0 2 – ASSESSMENT TOOL 3 – SOLUTION

NAME : M PRADEEPTHI

REGISTER NO : 192472387

COURSE CODE : CSA1704

COURSE NAME : ARTIFICIAL INTELLIGENCE

Question 1: Dry Run of A* Search Algorithm

Graph Summary

Edge	Cost
------	------

A–B	2
-----	---

A–C	4
-----	---

B–C	3
-----	---

B–D	7
-----	---

B–E	2
-----	---

D–E	2
-----	---

D–G	2
-----	---

E–G	2
-----	---

Heuristic values $h(n)$:

Node $h(n)$

A 7

B 6

C 4

D 3

E 2

G 0

Formula: $g(n) = f(n) = g(n) + h(n)$, where $g(n)$ = actual cost from start to n ,

$h(n) = h(n)$ = estimated cost from n to goal.

Iteration-by-Iteration Trace

Iteration 1

- **Current Node:** A
- **Open List:** [A]
- **Closed List:** []
- $g(A) = 0, h(A) = 7, f(A) = 7$

Expand A $\rightarrow$ neighbors B (cost 2), C (cost 4)

- $g(B) = 0 + 2 = 2, h(B) = 6, f(B) = 8$
- $g(C) = 0 + 4 = 4, h(C) = 4, f(C) = 8$

Move A to Closed List.

Iteration 2

- **Open List:** [B(f=8), C(f=8)]
- **Closed List:** [A]

f = 8 (tie). Both B and C have f = 8 (tie). By convention, pick alphabetically/first inserted → **Current Node: B**

- $g(B) = 2, h(B) = 6, \mathbf{f(B) = 8}$

Expand B → neighbors A(visited), C, D, E

- $g(C) \text{ via } B = 2 + 3 = 5 \rightarrow \text{worse than existing } g(C)=4, \text{ **discard**}$
- $g(D) = 2 + 7 = 9, h(D) = 3, f(D) = 12$
- $g(E) = 2 + 2 = 4, h(E) = 2, f(E) = 6$

Move B to Closed List.

Iteration 3

- **Open List:** [E(f=6), C(f=8), D(f=12)]
- **Closed List:** [A, B]
- **Current Node:** E (lowest f)
- $g(E) = 4, h(E) = 2, \mathbf{f(E) = 6}$

Expand E → neighbors B(visited), C, D, G

- $g(C) \text{ via } E = 4 + 3 = 7 \rightarrow \text{worse than 4, discard}$
- $g(D) \text{ via } E = 4 + 2 = 6 \rightarrow \text{better than 9! Update: } g(D) = 6, f(D) = 6 + 3 = 9$
- $g(G) = 4 + 2 = 6, h(G) = 0, f(G) = 6$

Move E to Closed List.

Iteration 4

- **Open List:** [G(f=6), C(f=8), D(f=9)]
- **Closed List:** [A, B, E]
- **Current Node:** G (lowest f)
- $g(G) = 6, h(G) = 0, f(G) = 6$

G is the goal node → stop.

Summary Table

Iteration	Current	g(n)	h(n)	f(n)	Open List	Closed List
1	A	0	7	7	[A]	{ }
2	B	2	6	8	[B, C]	{A}
3	E	4	2	6	[E, C, D]	{A, B}
4	G	6	0	6	[G, C, D]	{A, B, E}

Result

Optimal Path: $A \rightarrow B \rightarrow E \rightarrow G$

Cost breakdown: $A \rightarrow B (2) + B \rightarrow E (2) + E \rightarrow G (2)$

Cost = 2 + 2 + 2 = 6 Total Path Cost = 2 + 2 + 2 = 6

$h(G) = 0$, This matches $f(G) = 6$, confirming the path is optimal (since

$h(G) = 0$,

$g(G)$ at goal = actual total cost).

Question 2: Dry Run of Minimax with Alpha-Beta Pruning

Tree Structure



$\alpha = -\infty, \beta = +\infty$ Initial call: $\text{alphabeta}(\text{MAX}, \alpha = -\infty, \beta = +\infty)$

Step-by-Step Evaluation (Left to Right)

Branch 1: MIN1 (children 3, 5, 6)

$\alpha = -\infty, \beta = +\infty$ MIN1 starts with $\alpha = -\infty, \beta = +\infty$

Leaf 3: value = 3 $\rightarrow$ MIN1's current best (β) = $\min(+\infty, 3) = 3$

• $\beta(3) \leq \alpha(-\infty)$? No $\rightarrow$ continue Check pruning: $\beta(3) \leq \alpha(-\infty)$? No $\rightarrow$ continue

Leaf 5: value = 5 $\rightarrow$ MIN1's value = $\min(3, 5) = 3$ (no change, 5 is worse for MIN)

• β stays 3

• $\beta(3) \leq \alpha(-\infty)$? No $\rightarrow$ continue Check pruning: $\beta(3) \leq \alpha(-\infty)$? No $\rightarrow$ continue

Leaf 6: value = 6 $\rightarrow$ MIN1's value = $\min(3, 6) = 3$ (no change)

• β stays 3

MIN1 finishes all children $\rightarrow$ MIN1 = 3

Back at MAX: $\alpha = \max(-\infty, 3) = 3$

Branch 2: MIN2 (children 9, 1, 2)

$\alpha = 3$ MIN2 starts with $\alpha = 3$ (inherited from MAX), $\beta = +\infty$ **Leaf 9:** value = 9 $\rightarrow$ MIN2's value = $\min(+\infty, 9) = 9$

• $\beta(9) \leq \alpha(3)$? Check pruning: $\beta(9) \leq \alpha(3)$? No $\rightarrow$ continue **Leaf 1:** value = 1 $\rightarrow$ MIN2's value = $\min(9, 1) = 1$

- β updates to 1
- Check pruning: $\beta(1) \leq \alpha(3)$? **YES!** $\rightarrow$ **PRUNE remaining child (leaf value 2)**
- MIN2 stops here since MIN will never let this branch exceed 1, and MAX already has a guaranteed 3 elsewhere — so this branch is worthless to explore further.

MIN2 = 1 (final value, leaf "2" is never evaluated)

Back at MAX: $\alpha = \max(3, 1) = 3$ (no change, 3 is still better for MAX)

Alpha-Beta Table

Node	α (start)	β (start)	Value Computed	α (end)	β (end)	Pruned?
MIN1	$-\infty$	$+\infty$	$\min(3,5,6) = 3$	$-\infty$	3	No
MAX (after MIN1)	$-\infty$	$+\infty$	—	3	$+\infty$	—
MIN2	3	$+\infty$	9 $\rightarrow$ 1 (pruned after)	3	1	Yes
MAX (final)	3	$+\infty$	$\max(3, 1) = 3$	3	$+\infty$	—

Why the Pruning Happened

At MIN2, after seeing leaf 1, MIN2's value can only stay at 1 or go lower (MIN always picks the smallest). Since MAX already has a guaranteed score of 3 from MIN1, and MIN2 can never produce anything higher than 1 at this point, there is **no way MAX would ever choose MIN2's branch** over MIN1. So exploring the third leaf (value 2) under MIN2 is pointless — it's pruned.

This is a classic **beta cutoff** $\beta \leq \alpha$: $\beta \leq \alpha$ triggered inside a MIN node.

Result

- **Best Move for MAX:** Choose the **MIN1 branch** (value = 3)
- **Final Minimax Value:** 3
- **Pruned Nodes:** Leaf value 2 (the third child of MIN2) — never evaluated due to alpha-beta cutoff